



RMIT University

Further Programming

Student team: Group9

Student name & ID:

Bui Viet Anh s3988393

Ye Thu Aung s4066945

Jinho Ju s3807487

Lecturer: Minh Thanh Vu

Equipment Lending System (ELS)

I. Instruction for running programs

1.1 Prerequisites

- Java JDK 17 or newer.
- JavaFX SDK 17 or newer.
- Maven 3.6.0 or newer for dependency management and build automation.
- PostgreSQL Database setup remote via Supabase/Neon with required tables.
- Ensure Internet Connection is available for Maven to download dependencies.

1.2 Installation Steps

1. Clone the repository.
2. Download and extract JavaFX SDK to a permanent location on your computer.
3. JavaFX Environment Setup: Set up a PATH_TO_FX environment variable (optional but recommended):

- Windows (Command Prompt): set PATH_TO_FX=C:\path\to\javafx-sdk\lib
- macOS (Terminal - Bash): export PATH_TO_FX=/path/to/javafx-sdk/lib

Note: To make environment variable permanent on macOS, add the export line to your `~/.bash\_profile` file.

1.3 Build & Run the Application

Using Maven CLI (easiest method):

```
# Navigate to project root
- `cd further-programming-assignment-2-build-a-backend-tean9_sgs`

# Clean and build the project
- `mvn clean package`

# Run the application
- `mvn javafx:run`
```

Figure 1 Running Application Using Maven

Using JAR File:

After building:

- Windows:

```
# Run the generated JAR
- `java --module-path "C:\path\to\javafx-sdk\lib" --add-modules javafx.controls,
javafx.fxml,javafx.graphics -jar target\Assignmen-2-1.0-SNAPSHOT.jar`
Or
- `java --module-path %PATH_TO_FX% --add-modules javafx.controls,javafx.fxml,javafx.
graphics -jar target\Assignmen-2-1.0-SNAPSHOT.jar`
```

Figure 2 Running Application Using JAR File for Windows

- MacOS:

```
# Run the generated JAR
- `java --module-path "/path/to/javafx-sdk/lib" --add-modules javafx.controls,
javafx.fxml,javafx.graphics -jar target/Assignmen-2-1.0-SNAPSHOT.jar`
Or
- `java --module-path "$PATH_TO_FX" --add-modules javafx.controls,javafx.fxml,
javafx.graphics -jar target/Assignmen-2-1.0-SNAPSHOT.jar`
```

Figure 3 Running Application Using JAR File for MacOS

1.4 Program Flow

1. On launch, the application connects to the PostgreSQL database.
2. The Login Screen (JavaFX UI) appears for authentication.
3. Depending on the user role (Student, AcademicStaff, ProfessionalStaff, Administrator, Visitor), different dashboards are accessible.
4. Real-time data retrieval and CRUD operations on Equipments, Lending Records, and Courses are available.

1.5 Notes

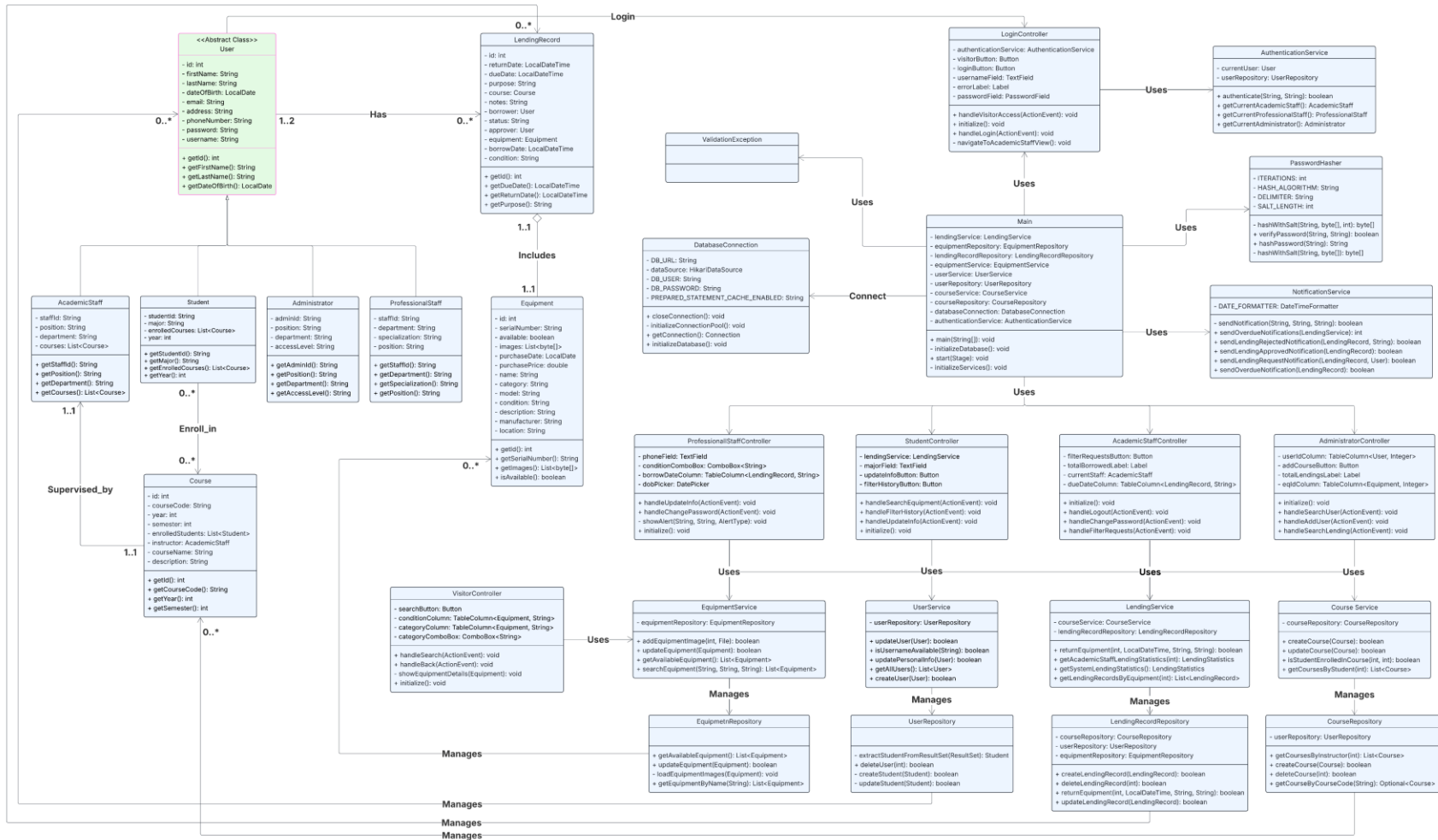
- Configuration for database URL, username, and password is in DatabaseConnection java file.
- For testing, use provided unit tests via:

```
- `mvn test`
```

Figure 4 Testing Application Command

- For GUI interaction, ensure system has graphical environment support.

Application UML Class Diagram



II. OOP Design

The design of the Equipment Lending System (ELS) is grounded in Object-Oriented Principles (OOP) to ensure a modular, scalable, and maintainable codebase. This section details the class structure, the design patterns employed, the application of core OOP principles, and the approach to exception handling.

3.1 Class Hierarchy and Relationships

The system's architecture is built upon a clear class hierarchy, promoting code reuse and logical organization. Relationships between classes are well-defined to model the domain accurately.

3.1.1 User Class Hierarchy

A central part of the ELS is its user management, which is modeled using an inheritance hierarchy:

- User (Abstract Class): This is the base class for all types of users within the system. It encapsulates common attributes such as user ID, name, and contact information, along with abstract methods like `getRole()` that must be implemented by subclasses. This abstraction provides a common interface for interacting with different user types.
 - Student: This class extends User and includes attributes specific to students, such as student ID and enrolled courses. It also incorporates functionality related to course enrollment and equipment borrowing specific to student users.
 - AcademicStaff: Extending User, this class caters to academic staff members. It includes attributes like staff ID and the courses they are responsible for. It provides functionalities for managing courses, approving student borrowing requests, and borrowing equipment for their own use.
 - ProfessionalStaff: This class also extends User and is designed for professional staff members. It includes relevant attributes such as employee ID and department. Their primary interaction with the lending system involves borrowing equipment for professional purposes.
 - Administrator: Extending User, this class represents users with the highest level of access and control over the system. It includes attributes defining their administrative

privileges and provides functionalities for managing all aspects of the ELS, including user accounts, equipment inventory, and system-wide settings.

3.1.2 Core Domain Classes

Beyond the user hierarchy, several other core classes model the primary entities of the equipment lending domain:

- **Course:** This class represents a university course. Key attributes include course code, course name, the AcademicStaff member assigned as the instructor, and a list of enrolled Student objects. It facilitates the association between students, academic staff, and equipment lending related to specific academic activities.
- **Equipment:** This class models a piece of equipment that can be borrowed. Attributes include equipment ID, name, description, category, condition (e.g., Good, Fair, Damaged), and availability status. It may also include methods for updating its status.
- **LendingRecord:** This class captures the details of an equipment borrowing transaction. It includes references to the User (borrower), the Equipment borrowed, the Course (if applicable, especially for student borrowings), the borrow date, due date, return date, and the AcademicStaff member who approved the request (if applicable). This class is crucial for tracking the lifecycle of equipment loans.

3.2 Design Patterns Used

To enhance the system's architecture, several established design patterns have been implemented:

1. **Model-View-Controller (MVC) Pattern:** This pattern separates the application into three interconnected components to manage complexity and improve maintainability.
 - **Model:** Represents the data and business logic of the application. This includes the domain classes: User, Student, AcademicStaff, ProfessionalStaff, Administrator, Course, Equipment, and LendingRecord.
 - **View:** Responsible for rendering the data from the Model into a user interface. In this system, the FXML files located in the resources directory constitute the View.
 - **Controller:** Acts as an intermediary between the Model and the View. It handles user input, interacts with the Model to perform operations, and updates the View to reflect

changes in the Model. Examples include LoginController, StudentController, and AcademicStaffController.

2. Repository Pattern: This pattern mediates between the domain and data mapping layers, acting like an in-memory collection of domain objects.
 - The system utilizes UserRepository, CourseRepository, EquipmentRepository, and LendingRecordRepository.
 - These repositories encapsulate the logic required to access data sources (e.g., a database). They provide a clean, object-oriented API to the service layer for CRUD (Create, Read, Update, Delete) operations, abstracting the underlying data storage mechanism.
3. Service Layer Pattern (Application Service Pattern): This pattern defines an application's boundary with a layer of services that establishes a set of available operations and coordinates the application's response in each operation.
 - Services like AuthenticationService, UserService, CourseService, EquipmentService, and LendingService are implemented.
 - These services encapsulate the business logic of the application. They orchestrate calls between controllers and repositories, ensuring that business rules are enforced and transactions are managed appropriately.
4. Factory Method Pattern: This creational pattern provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.
 - This pattern is employed within the repository classes. For instance, when retrieving user data from the database, a factory method can be used to instantiate the correct concrete user type (Student, AcademicStaff, etc.) based on the data, promoting flexibility in object creation.
5. Singleton Pattern: This creational pattern ensures that a class has only one instance and provides a global point of access to it.
 - The Singleton pattern is utilized for managing the database connection. This ensures that there is a single, shared connection pool or connection object throughout the application, optimizing resource usage and preventing connection management issues.

3.3 OOP Principles Applied

The design of the ELS adheres to fundamental OOP principles:

1. Inheritance: This principle allows new classes to be based on existing classes, inheriting their properties and methods.
 - The User class hierarchy is a prime example. Student, AcademicStaff, ProfessionalStaff, and Administrator all inherit from the abstract User class.
 - Each subclass extends the common functionality defined in User and adds attributes and behaviors specific to its role, promoting code reuse and a clear hierarchical structure.
2. Polymorphism: This principle allows objects of different classes to be treated as objects of a common superclass. It enables methods to behave differently based on the object that invokes them.
 - The abstract `getRole()` method in the User class is implemented differently by each concrete subclass (Student, AcademicStaff, etc.). This allows the system to call `getRole()` on any User object and get the specific role of that user type.
 - Type checking and casting are used in components like the AuthenticationService and various controllers to handle different user types appropriately, leveraging polymorphism to process user-specific logic through a common interface.
3. Encapsulation: This principle involves bundling the data (attributes) and the methods that operate on the data within a single unit (a class). It also restricts direct access to some of an object's components.
 - Class attributes are typically declared as private, with access provided through public getter and setter methods. This protects the internal state of objects and ensures data integrity.
 - Business logic is encapsulated within the service classes (e.g., CourseService), hiding the complexity from the controllers.
 - Data access logic is encapsulated within the repository classes, abstracting the details of database interactions from the service layer.
4. Abstraction: This principle involves hiding complex implementation details and exposing only the essential features of an object or system.
 - The abstract User class defines a common interface (a contract) for all user types, abstracting away the specific differences between Student, AcademicStaff, etc., when performing common user operations.

- Service interfaces can be used to hide the concrete implementation details of the service layer from the controllers, allowing for easier changes and testing.
- 5. Composition: This principle allows complex objects to be built from simpler ones. It represents a "has-a" relationship.
 - A Course object "has-a" list of enrolled Student objects and "has-an" AcademicStaff member as an instructor.
 - A LendingRecord object "has-a" reference to a borrower (User), an Equipment item, potentially a Course, and an approver (AcademicStaff). This allows LendingRecord to aggregate information from various related entities.

3.4 Exception Handling

A robust exception handling mechanism is crucial for building a reliable application. The ELS employs the following strategies:

- Custom ValidationException: A custom exception class, ValidationException, is defined to handle errors related to data validation (e.g., invalid input format, missing required fields). This allows for specific handling of validation issues.
- Try-Catch Blocks: Standard try-catch blocks are used extensively throughout the codebase, particularly in the service and repository layers, to handle potential SQLExceptions (for database interactions) and IOExceptions (for file operations, if any).
- User-Friendly Error Messages: When exceptions occur, they are caught and processed. Appropriate, user-friendly error messages are then displayed to the user through the UI (View layer). This prevents the application from crashing and provides meaningful feedback to the user about what went wrong.

This comprehensive OOP design provides a solid foundation for the Equipment Lending System, facilitating its development, maintenance, and future enhancements.

3.5 Future Development

Security Enhancements:

- Implement password hashing and salting for better security
- Add two-factor authentication for sensitive operations

- Implement session management and timeout features
- Add input validation and sanitization to prevent SQL injection

User Experience Improvements:

- Create a modern, responsive UI using CSS styling
- Add dark/light theme support
- Implement drag-and-drop functionality for equipment images
- Add real-time notifications for lending requests and updates
- Add bulk import/export functionality for equipment and user data
- Add in-app tutorials
- Implement a feedback system
- Live help or chatbot integration
- Multilingual interface supports

Feature Additions:

Implement email notifications for:

- Lending request status changes
- Due date reminders
- Overdue equipment alerts

Reporting and Analytics:

- Create detailed usage analytics and reports
- Implement equipment maintenance tracking
- Add predictive analytics for equipment demand
- Generate periodic usage reports for administrators

Database Optimizations:

- Add database indexing for better query performance
- Implement database migration tools
- Add database backup and restore functionality
- Implement data archiving for old records

Administrative Features:

- Add system configuration management interface
- Implement audit logging for all operations
- Create automated backup management

- Add role-based access control management

Equipment Management:

- Implement equipment maintenance scheduling
- Add equipment categorization and tagging
- Create equipment reservation system
- Implement equipment condition tracking